

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY LABORATORY INVESTIGATION & FORENSIC EVIDENCE REPORT

"Preparing Global Cybersecurity Leaders Through Excellence in Hands-On Training"

Lab Title: Digital Forensics Lab: USB Forensic Acquisition	Course Name: Digital Forensics
Student Name: Almustapha Yusuf	Instructor Name: Aminu Idris
Student ID: fwsd2511343	Date of Submission: 01/09/2026
Platform Environment: Kali Linux (Rolling Release)	Report Version: Version 1.0 (Final Release)

2.2 Executive Summary

This lab demonstrated the process of acquiring a forensic image of a USB drive. A simulated (virtual) USB drive was created and formatted as FAT32. An evidence folder was created containing a text file with student details. The virtual USB was acquired using the dd tool to create a raw forensic image. MD5 and SHA-256 hashes were calculated for both the source and the acquired image, which matched perfectly, confirming the acquisition's integrity. The acquired image was then mounted read-only to validate that the evidence file was present and readable. This lab highlights the importance of proper source identification, hashing, and image validation in digital forensics.

2.3 Lab Objectives

- Explain the purpose of forensic disk imaging in preserving digital evidence.
- Prepare controlled USB evidence using virtual block devices.
- Acquire a forensic raw USB image using the low-level Linux dd utility.
- Calculate dual cryptographic hash values (MD5 and SHA-256).
- Compare source and image hashes for forensic integrity verification.
- Validate the file system structure and contents of the acquired image under a read-only mount.
- Identify critical operational risks associated with low-level imaging tools.
- Document the complete acquisition process according to forensic standards.

2.4 Tools and Resources Used

- **Kali Linux:** Host operating platform providing the forensic execution suite and shell environment.
- **losetup:** Linux loop device manager used to bind raw disk image files as virtual block devices (/dev/loop0).
- **dd:** Low-level bitstream imaging utility used to perform byte-for-byte forensic duplication of block media.
- **md5sum / sha256sum:** Cryptographic utilities used to compute MD5 and SHA-256 digests for chain of custody verification.
- **mount / umount:** File system mounting utilities configured with read-only flags (-o loop,ro) for safe evidence validation.
- **dosfstools (mkfs.vfat):** MS-DOS / FAT32 file system manipulation utilities used to format simulated USB media.

2.5 Methodology

- **Part A (Preparation & Virtual USB Setup):** Established working workspace (/ICDFA-USB-Acquisition-Lab/{evidence,results,screenshots,scripts,reports}). Generated a 64MB blank binary container (virtual_usb.img) via dd from /dev/zero, formatted with a standard FAT32 file system using mkfs.vfat, bound to loopback device /dev/loop0, and mounted at /mnt/usb_sim. Created directory CIP-B102-Lab4-Evidence and populated it with evidence file Almustapha_Yusuf.txt.
- **Part B (Case Documentation & Forensic Metadata):** Recorded case parameters including Case ID (CIP-B102), Student Name (Almustapha Yusuf), Evidence Media (Simulated 64MB FAT32 USB), Acquisition Date (01/09/2026), Tool (dd with block size 4M, conv=noerror,sync), and Output Target (CIPB102_Lab4_Almustapha_Yusuf_USB.dd).
- **Part C (Forensic Acquisition):** Unmounted the active file system to maintain static bit consistency. Resolved loopback detachment by reattaching /dev/loop0 with partition scanning (losetup -fP), verified block geometry using lsblk, and acquired the forensic raw bitstream image using dd if=/dev/loop0 of=evidence/CIPB102_Lab4_Almustapha_Yusuf_USB.dd bs=4M status=progress conv=noerror,sync.
- **Part D (Cryptographic Hash Verification):** Computed MD5 and SHA-256 hashes independently for both the source container (virtual_usb.img) and the resulting forensic image (CIPB102_Lab4_Almustapha_Yusuf_USB.dd), piping values to results tracking logs and performing direct side-by-side verification.
- **Part E (Read-Only Evidence Mount & Content Validation):** Created dedicated mount point /mnt/verify, mounted the acquired raw image in strictly read-only mode (mount -o loop,ro), listed directory contents, verified the presence of Almustapha_Yusuf.txt, inspected its contents, and cleanly unmounted the evidence.

2.6 Screenshots and Evidence

The following documented figures depict the terminal execution, forensic acquisition, integrity hashing, and read-only validation performed inside the Kali Linux environment.

```
kali@kali: ~/ICDFA-USB-Acquisition-Lab
Session Actions Edit View Help

(kali@kali)-[~/ICDFA-Steganography-Lab]
$ cd ~
mkdir -p ICDFA-USB-Acquisition-Lab/{evidence,results,screenshots,scripts,reports}
cd ICDFA-USB-Acquisition-Lab

(kali@kali)-[~/ICDFA-USB-Acquisition-Lab]
$ which dosfstools
dosfstools not found

(kali@kali)-[~/ICDFA-USB-Acquisition-Lab]
$ sudo apt update && sudo apt install dosfstools -y
[sudo] password for kali:
Hit:1 http://http.kali.org/kali kali-rolling InRelease
2288 packages can be upgraded. Run 'apt list --upgradable' to see them.
dosfstools is already the newest version (4.2-1.2).
dosfstools set to manually installed.
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 2288

(kali@kali)-[~/ICDFA-USB-Acquisition-Lab]
$ # Create a blank 64MB file
dd if=/dev/zero of=virtual_usb.img bs=1M count=64

# Format it as FAT32 (standard for USB drives)
sudo mkfs.vfat virtual_usb.img
64+0 records in
64+0 records out
67108864 bytes (67 MB, 64 MiB) copied, 0.173624 s, 387 MB/s
mkfs.fat 4.2 (2021-01-31)

(kali@kali)-[~/ICDFA-USB-Acquisition-Lab]
$ # Attach the image to a loop device
sudo losetup -fP virtual_usb.img

# Find out which loop device it got assigned to (e.g., loop0, loop1)
lsblk
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
loop0 7:0 0 64M 0 loop
sda 8:0 0 80.1G 0 disk
└─sda1 8:1 0 80.1G 0 part /

(kali@kali)-[~/ICDFA-USB-Acquisition-Lab]
$ # Create a mount point
```

Figure 1: Environment Setup & Virtual USB Preparation (Part A) — Creating workspace, verifying dosfstools, allocating 64MB container, formatting as FAT32, and attaching /dev/loop0.

```
kali@kali: ~/ICDFA-USB-Acquisition-Lab
Session Actions Edit View Help
(kali@kali)-[~/ICDFA-USB-Acquisition-Lab]
$ # Create a mount point
sudo mkdir -p /mnt/usb_sim

# Mount the loop device
sudo mount /dev/loop0 /mnt/usb_sim

(kali@kali)-[~/ICDFA-USB-Acquisition-Lab]
$ sudo mkdir -p /mnt/usb_sim/CIP-B102-Lab4-Evidence

(kali@kali)-[~/ICDFA-USB-Acquisition-Lab]
$ sudo nano /mnt/usb_sim/CIP-B102-Lab4-Evidence/Almustapha_Yusuf.txt

(kali@kali)-[~/ICDFA-USB-Acquisition-Lab]
$ sudo cp ~/Downloads/*.jpg /mnt/usb_sim/CIP-B102-Lab4-Evidence/ 2>/dev/null
sudo cp ~/Downloads/*.pdf /mnt/usb_sim/CIP-B102-Lab4-Evidence/ 2>/dev/null

(kali@kali)-[~/ICDFA-USB-Acquisition-Lab]
$ # Unmount the virtual USB
sudo umount /mnt/usb_sim

# Detach the loop device (Optional - we can keep it attached for the next step)
sudo losetup -d /dev/loop0

(kali@kali)-[~/ICDFA-USB-Acquisition-Lab]
$ lsblk
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
sda 8:0 0 80.1G 0 disk
└─sda1 8:1 0 80.1G 0 part /

(kali@kali)-[~/ICDFA-USB-Acquisition-Lab]
$ cd ~/ICDFA-USB-Acquisition-Lab
sudo dd if=/dev/loop0 of=evidence/CIPB102_Lab4_Almustapha_Yusuf_USB.dd bs=4M status=progress conv=noerror,sync
0+0 records in
0+0 records out
0 bytes copied, 0.0015091 s, 0.0 kB/s

(kali@kali)-[~/ICDFA-USB-Acquisition-Lab]
$ sudo losetup -fP virtual_usb.img

(kali@kali)-[~/ICDFA-USB-Acquisition-Lab]
$ lsblk
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
```

Figure 2: *Evidence Ingestion, Initial Acquisition Error & Loopback Reattachment (Parts A & C)* — Mounting USB, writing Almustapha_Yusuf.txt, encountering 0-byte dd error upon detached loop device, and reattaching loopback.

```

(kali@kali)-[~/ICDFA-USB-Acquisition-Lab]
$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0       7:0    0   64M  0 loop
sda         8:0    0  80.1G  0 disk
└─sda1      8:1    0  80.1G  0 part /

(kali@kali)-[~/ICDFA-USB-Acquisition-Lab]
$ cd ~/ICDFA-USB-Acquisition-Lab
$ sudo dd if=/dev/loop0 of=evidence/CIPB102_Lab4_Almustapha_Yusuf_USB.dd bs=4M status=progress conv=noerror,sync
54525952 bytes (55 MB, 52 MiB) copied, 1 s, 50.3 MB/s
16+0 records in
16+0 records out
67108864 bytes (67 MB, 64 MiB) copied, 1.23441 s, 54.4 MB/s

(kali@kali)-[~/ICDFA-USB-Acquisition-Lab]
$ md5sum virtual_usb.img | tee results/hash_source_md5.txt
sha256sum virtual_usb.img | tee results/hash_source_sha256.txt
d13789627d9d5f6594117958098ed6e9  virtual_usb.img
aa1e8aa60bf02d2f66e3ae27eef0c788e308607e99c9ee83fd4dea7d2988c7b6  virtual_usb.img

(kali@kali)-[~/ICDFA-USB-Acquisition-Lab]
$ md5sum evidence/CIPB102_Lab4_Almustapha_Yusuf_USB.dd | tee results/hash_image_md5.txt
sha256sum evidence/CIPB102_Lab4_Almustapha_Yusuf_USB.dd | tee results/hash_image_sha256.txt
d13789627d9d5f6594117958098ed6e9  evidence/CIPB102_Lab4_Almustapha_Yusuf_USB.dd
aa1e8aa60bf02d2f66e3ae27eef0c788e308607e99c9ee83fd4dea7d2988c7b6  evidence/CIPB102_Lab4_Almustapha_Yusuf_USB.dd

(kali@kali)-[~/ICDFA-USB-Acquisition-Lab]
$ cat results/hash_source_md5.txt results/hash_image_md5.txt
cat results/hash_source_sha256.txt results/hash_image_sha256.txt
d13789627d9d5f6594117958098ed6e9  virtual_usb.img
d13789627d9d5f6594117958098ed6e9  evidence/CIPB102_Lab4_Almustapha_Yusuf_USB.dd
aa1e8aa60bf02d2f66e3ae27eef0c788e308607e99c9ee83fd4dea7d2988c7b6  virtual_usb.img
aa1e8aa60bf02d2f66e3ae27eef0c788e308607e99c9ee83fd4dea7d2988c7b6  evidence/CIPB102_Lab4_Almustapha_Yusuf_USB.dd

(kali@kali)-[~/ICDFA-USB-Acquisition-Lab]
$ sudo mkdir -p /mnt/verify
$ sudo mount -o loop,ro evidence/CIPB102_Lab4_Almustapha_Yusuf_USB.dd /mnt/verify

(kali@kali)-[~/ICDFA-USB-Acquisition-Lab]
$ ls -l /mnt/verify/CIP-B102-Lab4-Evidence/
cat /mnt/verify/CIP-B102-Lab4-Evidence/firstname_lastname.txt
total 1656

```

Figure 3: Forensic Acquisition & Dual Hash Verification (Parts C, D & E) — Successful dd acquisition (67MB copied), generating matching MD5/SHA-256 hashes, mounting read-only, and filename check.

```

(kali@kali)-[~/ICDFA-USB-Acquisition-Lab]
$ sudo mount -o loop,ro evidence/CIPB102_Lab4_Almustapha_Yusuf_USB.dd /mnt/verify
cat /mnt/verify/CIP-B102-Lab4-Evidence/Almustapha_Yusuf.txt
[sudo] password for kali:
Full Name: Almustapha Yusuf
Registration Number: fwsd2511343
Date: 01/09/2026
Statement: This USB was prepared for a controlled forensic acquisition lab.

(kali@kali)-[~/ICDFA-USB-Acquisition-Lab]
$ sudo umount /mnt/verify

(kali@kali)-[~/ICDFA-USB-Acquisition-Lab]
$

```

Figure 4: Evidence File Inspection & Clean Dismount (Part E) — Displaying contents of Almustapha_Yusuf.txt within read-only mounted image and cleanly unmounting /mnt/verify.

2.7 Analysis and Findings

A forensic comparison between the source virtual media and the acquired bitstream forensic image demonstrates complete bit-for-bit parity:

Forensic Artifact	Size	MD5 Hash Digest	SHA-256 Hash Digest
virtual_usb.img (Source Media)	67,108,8 64 bytes (64 MiB)	d13789627d9d5f65941179580 98ed6e9	aa1e8aa60bf02d2f66e3ae27eef0c788e308607e99c9ee83fd4 dea7d2988c7b6
CIPB102_Lab4_Almostapha_Yusuf _USB.dd (Acquired Forensic Image)	67,108,8 64 bytes (64 MiB)	d13789627d9d5f65941179580 98ed6e9	aa1e8aa60bf02d2f66e3ae27eef0c788e308607e99c9ee83fd4 dea7d2988c7b6

- **Bitstream Parity & Hash Verification:** The MD5 and SHA-256 digests computed on the acquired raw image perfectly matched those of the source container down to the exact bit. This mathematical verification confirms that no sector was modified, corrupted, or injected during the dd execution.
- **Read-Only Mount & Evidence Content Validation:** Mounting the raw forensic image under strict read-only parameters (-o loop,ro) at /mnt/verify permitted direct file system traversal without altering MAC (Modified, Accessed, Created) timestamps. The evidence file Almostapha_Yusuf.txt was present and fully intact.
- **Controlled Content Integrity:** Displaying the contents of the recovered text file confirmed all original evidence entries: Full Name (Almostapha Yusuf), Registration Number (fwsd2511343), Date (01/09/2026), and Statement of forensic lab preparation.

2.8 Challenges and Solutions

- **Challenge 1 (Loop Device Premature Detachment):** When attempting the initial acquisition with dd if=/dev/loop0 of=evidence/CIPB102_Lab4_Almostapha_Yusuf_USB.dd, the command output showed '0+0 records in / 0+0 records out / 0 bytes copied'. This occurred because the loop device /dev/loop0 had been unmounted and detached via losetup -d /dev/loop0 prior to the acquisition command.
- **Solution 1:** Reattached the raw disk image virtual_usb.img to the loopback controller using sudo losetup -fP virtual_usb.img. Confirmed its presence and size (64M) using lsblk, and re-executed the dd acquisition command, resulting in a successful transfer of 16+0 records (67,108,864 bytes).
- **Challenge 2 (Evidence Filename Mismatch during Validation):** During the read-only post-acquisition validation phase, an initial attempt to read the evidence using cat /mnt/verify/CIP-B102-Lab4-Evidence/firstname_lastname.txt failed because the command referenced a template placeholder name.
- **Solution 2:** Executed directory listing (ls -l /mnt/verify/CIP-B102-Lab4-Evidence/) to inspect the actual directory structure. Located the correct file name (Almostapha_Yusuf.txt) and executed the cat command, successfully displaying the evidence statement.

2.9 Conclusion

This lab successfully demonstrated the industry-standard forensic acquisition process using the low-level dd utility. By capturing a physical block-level bitstream image, all active files, file system metadata, partition tables, and unallocated clusters were preserved identically. The strict correlation of both MD5 and SHA-256 cryptographic digests proved that the forensic duplicate was a bit-for-bit match of the source media. Furthermore, mounting the image read-only demonstrated a safe, non-destructive methodology for forensic validation. The challenges encountered emphasized the critical nature of device identification and state management when executing low-level disk operations.

2.10 Recommendations

- **Verify Source and Destination Identifiers via lsblk:** Always verify block device names (e.g., /dev/loop0, /dev/sdb) prior to executing dd. Inverting the input file (if=) and output file (of=) parameters will result in catastrophic, irreversible data loss on the suspect evidence.
- **Implement Hardware or Software Write-Blockers:** In operational investigations, physical USB media must always be attached via certified hardware write-blockers or software write-blocking flags to prevent operating systems from writing hidden system files or modifying volume timestamps.
- **Maintain Dual-Hash Chain of Custody Documentation:** Forensic examiners should always record both MD5 and SHA-256 cryptographic digests in formal acquisition logs to protect against hash collision vulnerabilities and maintain strict legal defensibility in court proceedings.

2.11 References

- Carrier, B. (2020). The Sleuth Kit (TSK) & Autopsy Documentation. Retrieved from <https://www.sleuthkit.org/>
- Kerrisk, M. (2022). dd(1) — Linux manual page. Linux Programmer's Manual. Retrieved from <https://man7.org/linux/man-pages/man1/dd.1.html>
- National Institute of Standards and Technology (NIST). (2021). Guide to Digital Forensics (Special Publication 800-86). U.S. Department of Commerce.
- International Cybersecurity and Digital Forensics Academy (ICDFA). (2026). Lab Assignment Template Guide & USB Forensic Acquisition Courseware.